

C# Threading Overview

C# provides robust support for multithreading, allowing developers to create and manage multiple threads within an application. This is essential for performing tasks concurrently, improving application responsiveness, and utilizing system resources efficiently.

Key Concepts

1. Thread Class:

- Represents a thread of execution in a program.
- Can be created using the `Thread` class from the `System.Threading` namespace.

2. ThreadStart Delegate:

- A delegate that represents a method that takes no parameters and does not return a value.
- Used to specify the method that will run on a new thread.

```
csharp
VerifyOpen In EditorRunCopy code
1public delegate void ThreadStart();
```

3. ParameterizedThreadStart Delegate:

- A delegate that represents a method that takes a single parameter of type `object` and does not return a value.
- Useful for passing data to the thread when it starts.

```
csharp
VerifyOpen In EditorRunCopy code
1public delegate void ParameterizedThreadStart(object obj);
```

4. ThreadPool:

- A pool of worker threads managed by the .NET runtime.
- Allows for efficient use of threads by reusing them for multiple tasks, reducing the overhead of thread creation and destruction.

Creating Threads

Using ThreadStart

To create a thread using the `ThreadStart` delegate, you can define a method that does not take any parameters:

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading;
3
4public class Program
5{
6    public static void Main()
7    {
8        Thread thread = new Thread(new ThreadStart(DoWork));
```

```

9      thread.Start();
10  }
11
12  public static void DoWork()
13  {
14      Console.WriteLine("Thread is working...");
15      Thread.Sleep(2000); // Simulate work
16      Console.WriteLine("Thread work completed.");
17  }
18}

```

Using ParameterizedThreadStart

To pass parameters to a thread, use the `ParameterizedThreadStart` delegate:

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading;
3
4public class Program
5{
6    public static void Main()
7    {
8        Thread thread = new Thread(new ParameterizedThreadStart(DoWork));
9        thread.Start("Hello from the thread!");
10    }
11
12    public static void DoWork(object message)
13    {
14        Console.WriteLine(message);
15        Thread.Sleep(2000); // Simulate work
16        Console.WriteLine("Thread work completed.");
17    }
18}

```

Using ThreadPool

The `ThreadPool` class provides a way to execute tasks on a pool of threads. This is particularly useful for short-lived tasks:

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading;
3
4public class Program
5{
6    public static void Main()
7    {
8        ThreadPool.QueueUserWorkItem(DoWork, "Hello from ThreadPool!");
9        Console.WriteLine("Main thread is doing other work...");
10        Thread.Sleep(1000); // Simulate work
11        Console.WriteLine("Main thread work completed.");
12    }
13
14    public static void DoWork(object state)
15    {
16        Console.WriteLine(state);
17        Thread.Sleep(2000); // Simulate work
18        Console.WriteLine("ThreadPool work completed.");
19    }

```

Synchronizing Critical Data

In C#, synchronizing access to critical data in a multithreaded environment is essential to prevent race conditions and ensure thread safety. Two common mechanisms for achieving this are the `Monitor` class and the `Interlocked` class.

1. Using the Monitor Class

The `Monitor` class provides a way to synchronize access to a block of code, ensuring that only one thread can execute that block at a time. This is often done using the `lock` statement.

Example Code:

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading;
3
4public class Account
5{
6    private readonly object _balanceLock = new object();
7    private decimal _balance;
8
9    public Account(decimal initialBalance) => _balance = initialBalance;
10
11    public decimal Debit(decimal amount)
12    {
13        if (amount < 0)
14            throw new ArgumentOutOfRangeException(nameof(amount), "The debit
amount cannot be negative.");
15
16        decimal appliedAmount = 0;
17        lock (_balanceLock)
18        {
19            if (_balance >= amount)
20            {
21                _balance -= amount;
22                appliedAmount = amount;
23            }
24        }
25        return appliedAmount;
26    }
27
28    public void Credit(decimal amount)
29    {
30        if (amount < 0)
31            throw new ArgumentOutOfRangeException(nameof(amount), "The credit
amount cannot be negative.");
32
33        lock (_balanceLock)
34        {
35            _balance += amount;
36        }
37    }
38
39    public decimal GetBalance()
40    {
41        lock (_balanceLock)
42    {
```

```

43         return _balance;
44     }
45 }
46}
47
48class Program
49{
50     static void Main()
51     {
52         var account = new Account(1000);
53         Parallel.For(0, 100, i =>
54         {
55             account.Credit(10);
56             account.Debit(5);
57         });
58         Console.WriteLine($"Account's balance is {account.GetBalance()}");
59     }
60}

```

2. Using the Interlocked Class

The `Interlocked` class provides atomic operations for variables shared by multiple threads. It is particularly useful for simple operations like incrementing or decrementing a variable without the overhead of a lock.

Example Code:

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading;
3
4public class Counter
5{
6    private int _count;
7
8    public void Increment()
9    {
10        Interlocked.Increment(ref _count);
11    }
12
13    public int GetCount()
14    {
15        return _count;
16    }
17}
18
19class Program
20{
21    static void Main()
22    {
23        var counter = new Counter();
24        Parallel.For(0, 1000, i =>
25        {
26            counter.Increment();
27        });
28        Console.WriteLine($"Final count is {counter.GetCount()}");
29    }
30}

```

Working with Tasks

In C#, the `async` and `await` keywords are used to simplify asynchronous programming. They allow you to write code that performs asynchronous operations without blocking the main thread.

1. Async Methods with Return Values

When you have an asynchronous method that returns a value, you typically use `Task<TResult>` as the return type.

Example: Async Method Returning a Value

```
csharp
VerifyOpen In EditorRunCopy code
1public async Task<int> GetLeisureHoursAsync()
2{
3    await Task.Delay(1000);
4    DayOfWeek today = DateTime.Now.DayOfWeek;
5    return (today == DayOfWeek.Saturday || today == DayOfWeek.Sunday) ? 16 : 5;
6}
7
8public async Task ShowLeisureHoursAsync()
9{
10    int leisureHours = await GetLeisureHoursAsync();
11    Console.WriteLine($"Today's leisure hours: {leisureHours}");
12}
```

2. Async Methods without Return Values

For asynchronous methods that do not return a value, you use `Task` as the return type.

Example: Async Method Without Return Value

```
csharp
VerifyOpen In EditorRunCopy code
1public async Task WaitAndApologizeAsync()
2{
3    await Task.Delay(2000);
4    Console.WriteLine("Sorry for the delay...");
5}
6
7public async Task DisplayCurrentInfoAsync()
8{
9    await WaitAndApologizeAsync();
10    Console.WriteLine($"Today is {DateTime.Now:D}");
11}
```

3. Calling Async Methods

When calling async methods, you should always use `await` to ensure that the calling method waits for the async operation to complete.

Example: Calling Async Methods

```
csharp
VerifyOpen In EditorRunCopy code
1public async Task ExecuteAsync()
2{
3    await ShowLeisureHoursAsync();
4    await DisplayCurrentInfoAsync();
5}
```

Task Parallel Library (TPL) Overview

The **Task Parallel Library (TPL)** is a set of public types and APIs in the `System.Threading` and `System.Threading.Tasks` namespaces designed to simplify the process of adding parallelism and concurrency to applications.

Key Features of TPL

- **Higher-Level Abstraction:** TPL abstracts the complexities of thread management.
- **Dynamic Scalability:** Automatically scales the degree of concurrency based on available processors.
- **Task-Based Model:** Tasks are lightweight and can be easily created, managed, and composed.
- **Cancellation Support:** Provides mechanisms to cancel tasks gracefully.
- **Exception Handling:** Offers structured ways to handle exceptions that occur in parallel tasks.

Examples of Using TPL

1. Creating and Running a Basic Task

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading.Tasks;
3
4class Program
5{
6    static void Main()
7    {
8        Task myTask = Task.Run(() => {
9            Console.WriteLine("Hello from the task!");
10        });
11
12        myTask.Wait(); // Wait for the task to complete
13    }
14}
```

2. Task with Cancellation

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading;
3using System.Threading.Tasks;
4
5class Program
6{
7    static void Main()
8    {
9        var cts = new CancellationTokenSource();
10        var token = cts.Token;
11
12        Task task = Task.Run(() => {
13            while (!token.IsCancellationRequested)
14            {
15                Console.WriteLine("Task is running");
16                Thread.Sleep(1000);
17            }
18        });
19    }
20}
```

```

18         token.ThrowIfCancellationRequested();
19     }, token);
20
21     Thread.Sleep(3000);
22     cts.Cancel();
23     try
24     {
25         task.Wait();
26     }
27     catch (AggregateException ae)
28     {
29         if (ae.InnerExceptions[0] is OperationCanceledException)
30         {
31             Console.WriteLine("Task was cancelled.");
32         }
33     }
34 }
35}

```

3. Parallel Processing with Parallel.ForEach

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Collections.Generic;
3using System.Threading.Tasks;
4
5class Program
6{
7    static void Main()
8    {
9        var numbers = new List<int> { 1, 2, 3, 4, 5 };
10
11        Parallel.ForEach(numbers, number =>
12        {
13            Console.WriteLine($"Processing number {number} on thread
14{Thread.CurrentThread.ManagedThreadId}");
15        });
16}

```

4. Handling Exceptions in TPL

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading.Tasks;
3
4class Program
5{
6    static void Main()
7    {
8        try
9        {
10            Task task = Task.Run(() => { throw new Exception("Error in
11task."); });
12            task.Wait();
13        }
14        catch (AggregateException ae)
15        {
16            foreach (var e in ae.InnerExceptions)
17            {
18                Console.WriteLine($"Handled exception: {e.Message}");
19            }
20        }
21    }
22}

```

```
18     }
19 }
20 }
21}
```

5. Using Task Continuations

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading.Tasks;
3
4class Program
5{
6    static void Main()
7    {
8        Task initialTask = Task.Run(() => {
9            Console.WriteLine("Initial task running.");
10           return 42;
11        });
12
13        initialTask.ContinueWith(t => {
14            Console.WriteLine($"Continuation task running. Result:
15{t.Result}");
16        }, TaskContinuationOptions.OnlyOnRanToCompletion);
17}
```

Conclusion

This comprehensive overview covers the essential concepts of threading, synchronization, asynchronous programming, and the Task Parallel Library in C#. Understanding these concepts will help you write efficient, responsive, and thread-safe applications. For further exploration, consider diving into more advanced topics such as thread synchronization techniques, error handling, and performance optimization strategies.